

Milestone 4 — Honey Yield Predictive Model

Team: Stephanie Nord, David Jorgensen, Joshua Amaya **Course project:** predicting next-day hive weight change from colony sensor history

How to run this notebook

```
pip install -r requirements.txt
jupyter nbconvert --to html --execute Milestone_4.ipynb
```

This notebook runs reproducibly from the project repository using the committed `data/honey_model.parquet` dataset, without reliance on local database files, credentials, or machine-specific paths.

Work is split between two paths, and the distinction matters when reading the numbers. The result tables in Sections 2–6 are produced by `scripts/run_evaluation.py` and `scripts/run_extremes_analysis.py`, written to `results/`, and read back here — they are only as current as the last run of those scripts. The figures, the fold construction, the model refit in Section 5.2, the gate ablation in Section 5.3, the leave-hives-out baseline in Section 5.5, and the segmented report in Section 6.1 are computed when the notebook executes. Both paths read only the committed Parquet file.

The reproducibility workflow was strengthened for Milestone 4 following feedback from the previous milestone. The current implementation provides a consistent shared data source that allows the complete analytical workflow to be reproduced across team members and computing environments.

What this milestone delivers

- 1. **A documented data-preparation process** (Section 2), including a row-count waterfall that documents observations removed during construction of the modeling dataset.
- 2. **Model development and evaluation** (Section 5), including naive baselines, chronological evaluation, rolling-origin cross-validation, and evaluation of a two-stage modeling approach.
- 3. **Interpretation of results** (Section 6), including additional analyses that refine findings from Milestone 3.
- 4. **Preliminary conclusions and recommendations** (Section 7), incorporating the expanded analysis conducted during this milestone.

How Milestone 4 builds on Milestone 3

Milestone 3 established the initial end-to-end daily modeling pipeline and identified several findings that warranted additional investigation, including the effect of extreme weight changes on model performance and the potential relationship between those observations and beekeeper activity.

For Milestone 4, the pipeline was scaled and made fully reproducible using the complete daily dataset. The evaluation framework was also strengthened through additional baselines, chronological validation, rolling-origin cross-validation, and further examination of extreme observations and event-related variables. These analyses refine several preliminary findings from Milestone 3 rather than replacing the original modeling approach.

The following table summarizes how the expanded Milestone 4 analysis refined the preliminary findings reported in Milestone 3.

Milestone 3 finding	Milestone 4 refinement	Section
Routine-only modeling produced $R^2 = 0.216$ under the initial evaluation approach.	Expanded evaluation showed that performance varies depending on how routine observations are defined and evaluated, emphasizing the importance of evaluating models on the full deployment population.	5.3
Extreme weight changes showed limited association with the initially examined beekeeper-event variables.	The publisher's own rate-based filter removes 419 of the 445 extremes, which is the strongest available evidence for a handling-related component; the logged beekeeper-event variables themselves remain largely uninformative.	3.5
Initial event-distance analysis identified few extremes occurring	The source fields mix days and seconds across hives, so the original figures were understated. After normalization and correction of the denominator, extremes are	3.3

Milestone 3 finding	Milestone 4 refinement	Section
within one day of logged events.	still not meaningfully closer to logged events than routine days; feeding is the one category differing in the expected direction, and only marginally.	
Gradient Boosting outperformed Random Forest in the initial comparison.	Expanded evaluation across folds and random seeds provided a more robust comparison of the two ensemble approaches.	5.5

1. Setup

```
In [1]: import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn

sys.path.insert(0, str(Path.cwd() / "src"))

from honeymodel import data, evaluation as ev, features, models

warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 180)
pd.set_option("display.max_columns", 40)
plt.rcParams.update({"figure.dpi": 110, "axes.grid": True, "grid.alpha": 0.3})

RESULTS = Path("results")

def show(name, **kwargs):
    # Render a table produced by scripts/run_*.py.
    return pd.read_csv(REULTS / name, **kwargs)

# Model fits below are seeded and deterministic within a fixed environment, but tree
# implementations change between library releases. The versions that produced the stored
# outputs are printed so a future re-run that disagrees can be traced to the cause.
print(f"python {sys.version.split()[0]} scikit-learn {sklearn.__version__} "
      f"pandas {pd.__version__} numpy {np.__version__}")

model_df = data.add_season(data.load_model_table())
print(f"modelling rows : {len(model_df):,}")
print(f"hives          : {model_df.hive_id.nunique()}")
print(f"date range       : {model_df.measurement_date.min().date()} to {model_df.measurement_date.max().date}")
print(f"columns          : {model_df.shape[1]}")

python 3.13.5 scikit-learn 1.6.1 pandas 2.2.3 numpy 2.1.3
modelling rows : 26,215
hives          : 78
date range     : 2019-06-25 to 2022-12-30
columns        : 72
```

`load_model_table` asserts the published shape on every load — 26,215 rows across 78 hives, no duplicate hive-days — so a silently altered snapshot cannot reach the analysis below.

2. Data Preparation Process

2.1 Source and provenance

Senger, D., Gruber, C., Kluss, T., & Johannsen, C. (2024). Weight, temperature and humidity sensor data of honey bee colonies in Germany, 2019–2022. *Data in Brief*, 52, 110015.

<https://doi.org/10.1016/j.dib.2023.110015>

Dataset: <https://doi.org/10.5281/zenodo.10407693>

78 colonies across Germany, June 2019 – December 2022, instrumented under a citizen-science project. Each hive carries five internal temperature sensors, an external temperature sensor, a combined temperature/humidity/pressure sensor, and a scale. Beekeepers logged inspections and interventions through the project's own web app.

Dataset provenance clarification. During Milestone 3, the primary dataset was described as “HOBOS,” and the observation logs were attributed to “USDA Tucson.” As the project expanded to the complete daily archive for Milestone 4, further review of the source documentation clarified that the primary data are from the German citizen-science project described by Senger et al. (2024), and the beekeeper observation logs originate from that project. The source information and citation have been updated accordingly.

Licensing. The Zenodo record is open access but declares no license — the DataCite `rightsList` is empty. We commit a derived daily-grain table (about 0.05% of the 5.6 GB archive) with full attribution so that this notebook can be reproduced without credentials, and have written to the corresponding author to confirm the intended terms. `data/README.md` records the fallback if redistribution turns out to be restricted.

2.2 Publisher preprocessing

Understanding the preprocessing performed by the dataset publishers is important for interpreting the derived weight variables and subsequent modeling results. From the paper’s methods:

- **Range filters.** Weights above 150 kg or below −50 kg, temperatures above 85 °C or below −40 °C, and humidity outside 0–100% were excluded.
- **Rate filter.** The weight series was differentiated and changes above **0.3 kg per minute** excluded, because such “sudden drastic changes in the weight [...] are usually induced by activities by the beekeeper.”
- **Aggregation.** Raw readings arrive every 5–10 seconds; the minute series is a median, and the hourly and daily series are averages of the minute series.

Two consequences matter a great deal, and Section 3.5 quantifies the first:

1. The rate filter produced a **second weight column**. `weight_kg` keeps every jump; `weight_kg_noOutlier` is the cumulative sum of the same deltas with flagged jumps zeroed. Milestone 3 built its target from `weight_kg` — the series that still contains the events the publishers attribute to beekeeper handling.
2. `end_of_day_weight_kg` is a **misnomer**: the daily value is the mean of the day’s minute readings, not the last one. The target is a change in daily *mean* weight. The column name is retained for continuity with the Milestone 3 schema and is corrected in the documentation.

2.3 Ingestion and grain

The publisher’s daily files come in two arrangements, and only one of them is a genuine archive of the whole record:

dataset	files	rows	hives	what it is
years	78	29,172	78	the full 2019–2022 daily archive
events	191	12,538	55	3-month windows either side of a swarming or colony-death event

The `events` files re-slice hive-days that `years` already contains: only 948 of its hive-days (3.2%) are new, and it holds 527 internal duplicates. The pipeline ingests `years` only.

Milestone 3 developed and validated the initial pipeline using the available subset of daily files. For Milestone 4, the workflow was expanded to the complete `years` archive, producing 29,172 daily observations across all 78 hives. The hive count, temporal coverage, and 1.7% extreme-event rate reported below therefore reflect the complete daily archive.

Typing uses `TRY_CAST` with `NULLIF(col, 'NA')` throughout (`sql/02_clean.sql`), and the grain — one row per hive per day — is asserted in `scripts/build_honey_model.py` *before* any window function runs, rather than assumed.

2.4 Filtering: a row-count waterfall

```
In [2]: waterfall = show("data_preparation_waterfall.csv")
display(waterfall)
print(f"total rows dropped: {waterfall.rows_dropped.sum():,} "
      f"({100 * waterfall.rows_dropped.sum() / waterfall.rows_remaining.iloc[0]:.1f}% of ingested rows)")
```

	gate	rows_remaining	rows_dropped
0	daily rows ingested (years, interval=d)	29172	0
1	has a previous consecutive day	28078	1094
2	has a next consecutive day	27110	968
3	all three weights present	27092	18
4	all three weights > 0 (Milestone 3 guard)	26215	877

total rows dropped: 2,957 (10.1% of ingested rows)

The row-count waterfall documents the reduction from 29,172 daily observations to 26,215 modeling observations. Of the 2,957 excluded rows, 1,094 lack a consecutive previous-day observation, 968 lack a consecutive next-day observation, 18 are missing one of the three required weight values, and 877 do not satisfy the positive-weight criterion.

2.5 Missingness

```
In [3]: sensor_columns = features.SENSOR_FEATURES
missing = features.summarise_missingness(model_df, sensor_columns)
display(missing)
```

	column	missing	missing_pct
0	avg_humidity	9271	35.37
1	avg_pressure	8118	30.97
2	avg_internal_temp_4	7328	27.95
3	avg_internal_temp_1	6610	25.21
4	avg_internal_temp_3	6538	24.94
5	avg_outside_temp	6081	23.20
6	avg_internal_temp_2	6066	23.14

The sensor variables contain substantial missingness, ranging from approximately 23% to 35% across the modeling dataset. Because removing all observations with missing sensor values would result in considerable data loss, the modeling workflow uses approaches that preserve observations while preventing imputation from introducing information across independent hives.

For models requiring complete feature values, forward-fill imputation is performed independently within each hive after observations are ordered chronologically by `measurement_date`. Restricting imputation by `hive_id` prevents measurements from one colony from being propagated into another colony. Missingness indicators are also retained for imputed sensor variables, allowing the models to distinguish observed measurements from values introduced through imputation.

When supported by the modeling algorithm, missing values are retained rather than imputed.

`HistGradientBoostingRegressor` and `LightGBM` can handle missing values natively, allowing these models to use the available sensor information without requiring the removal of incomplete observations or additional imputation. Together, these approaches preserve as much of the available data as possible while maintaining separation between hives and reducing the potential for artifacts introduced during preprocessing.

2.6 Leakage controls

```
In [4]: # honey_model deliberately ships columns that are only knowable after the prediction date
# -- the extreme-event investigation needs them. They must never reach a feature matrix.
print(f"{len(features.FORBIDDEN_COLUMNS)} forbidden columns, including:")
for column in sorted(features.FORBIDDEN_COLUMNS)[:6]:
    print(f" {column}")

# Negative test: asking for one must fail, not silently produce a perfect model.
try:
    features.build_feature_matrix(model_df, feature_set=["previous_day_weight_kg", "next_day_weight_kg"])
```

```
except features.LeakageError as error:
    print(f"\nLeakageError raised as intended:\n {error}")
```

24 forbidden columns, including:

- days_to_next_observation
- died_next_dif
- extreme_relative_flag
- extreme_weight_change_flag
- extreme_weight_change_flag_10
- extreme_weight_change_flag_3

LeakageError raised as intended:

these columns are only knowable after the prediction date, or are derived from the label, and must not be used as features: ['next_day_weight_kg']

Because `next_day_weight_kg` and other post-prediction variables contain information that would not be available at prediction time, these variables are explicitly excluded from the feature matrix to prevent target leakage. The modeling pipeline programmatically enforces this restriction, and a negative test confirms that prohibited variables cannot be incorporated as predictors. This safeguard helps ensure that reported model performance reflects information that would realistically be available when generating a next-day prediction.

3. Data Quality and Extreme-Event Analysis

Milestone 3 separated the target distribution into routine observations (98.3%) and extreme weight changes (1.7%; $|\Delta| > 5$ kg, 445 observations), demonstrating that a relatively small number of extreme observations substantially influence model performance. This distinction remains an important component of the Milestone 4 analysis.

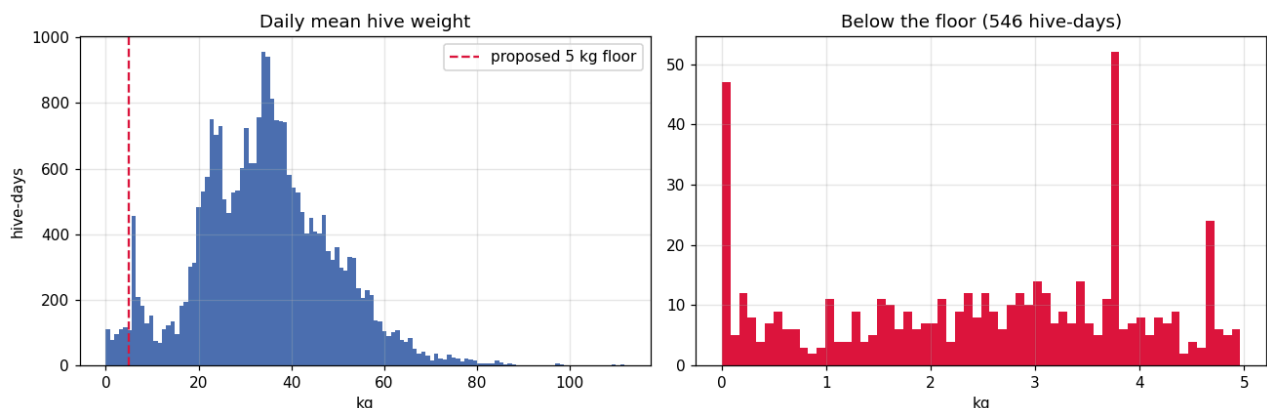
For Milestone 4, the extreme-event investigation was aligned directly with the 445 daily observations identified in the final modeling dataset. Earlier exploratory analysis also examined publisher-defined outlier indicators at the minute-level grain; however, those indicators represent a different observation frequency and outlier definition than the daily prediction target. The analyses below therefore focus specifically on the daily extreme observations used in the predictive modeling workflow.

3.1 Physically implausible weights

```
In [5]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(model_df.end_of_day_weight_kg, bins=120, color="#4c72b0", edgecolor="none")
axes[0].axvline(5, color="crimson", linestyle="--", label="proposed 5 kg floor")
axes[0].set(title="Daily mean hive weight", xlabel="kg", ylabel="hive-days")
axes[0].legend()

low = model_df[model_df.end_of_day_weight_kg < 5]
axes[1].hist(low.end_of_day_weight_kg, bins=60, color="crimson", edgecolor="none")
axes[1].set(title=f"Below the floor ({len(low):,} hive-days)", xlabel="kg")
plt.tight_layout(); plt.show()

print(f"minimum weight in the modelling table : {model_df.end_of_day_weight_kg.min():.4f} kg")
print(f"mean weight                               : {model_df.end_of_day_weight_kg.mean():.1f} kg")
print(f"rows flagged implausible                   : {int(model_df.implausible_weight_flag.sum()):,}")
```



```
minimum weight in the modelling table : 0.0005 kg
mean weight                           : 34.3 kg
rows flagged implausible               : 597
```

A transition from a typical hive weight to a near-zero reading can generate a very large calculated daily weight change that may reflect measurement or processing artifacts rather than biological colony change.

3.2 Data-quality characteristics of extreme observations

```
In [6]: display(show("extremes_attribution.csv"))
```

	explanation	extremes_flagged	pct_of_extremes	total_extremes
0	implausible weight (below 5 kg floor)	19	4.27	445
1	hive-local robust outlier (>5 MAD)	122	27.42	445
2	drop-then-recovery (dropout / re-tare)	26	5.84	445
3	any data-quality flag	138	31.01	445

The diagnostic analysis identified at least one data-quality flag for 138 of the 445 extreme observations (31.0%). Of the extreme observations, 4.3% involved hive weights below the proposed 5 kg plausibility threshold, 27.4% were identified as hive-local robust outliers using the >5 MAD criterion, and 5.8% exhibited a drop-then-recovery pattern consistent with a possible sensor dropout or scale re-tare. Because these diagnostic categories may overlap, they should not be interpreted as mutually exclusive causes of extreme weight change. The remaining observations were not identified by the data-quality criteria evaluated here; however, the available evidence is insufficient to attribute those observations to a specific biological, environmental, or beekeeper-related mechanism. These results indicate that data-quality anomalies contribute to a meaningful subset of extreme observations and should be considered when interpreting model performance.

3.3 Proximity to logged beekeeper events — and a unit defect in the source

```
In [7]: display(show("event_distance_units.csv"))
```

	column	hives_in_days	hives_in_seconds	hives_undetermined
0	honey_last_dif	21	16	41
1	honey_next_dif	4	20	54
2	feeding_last_dif	25	16	37
3	feeding_next_dif	3	29	46
4	swarming_last_dif	6	7	65
5	swarming_next_dif	8	7	63
6	treatment_last_dif	22	5	51
7	treatment_next_dif	7	16	55
8	died_last_dif	7	3	68
9	died_next_dif	3	3	72
10	queencell_last_dif	9	9	60
11	queencell_next_dif	5	8	65

The published *_last_dif and *_next_dif variables describe the temporal distance between sensor observations and logged beekeeper events. Examination of consecutive daily observations indicated that these variables are not represented in a consistent unit across the dataset. For some records, the values advance by approximately 1 per day, whereas others advance by approximately 86,400, consistent with seconds per day. Consequently, direct threshold comparisons applied to the unstandardized variables may underestimate event proximity for records represented in seconds.

To improve comparability, the Milestone 4 workflow evaluates the units represented within these event-distance variables and converts identifiable values to a common daily scale. This analysis also revealed that unit representation may change within individual hive histories, meaning that per-hive normalization does not completely resolve the inconsistency.

How much the table above actually resolves — not much. The three columns partition the 78 hives, and for every column the undetermined group is the largest of the three. A hive is undetermined when it never contributes two consecutive days with that column populated, which leaves no increment from which to infer a unit. Only 3 to 25 hives per column are positively identified as day-scaled; `died_next_dif`, for instance, resolves 3 hives as days and 3 as seconds against 72 undetermined.

Undetermined hives are left unconverted, so any that are in fact seconds-scaled still cannot register as near an event. The conversion therefore corrects a defect it can only partially detect, and the proximity estimates in Section 3.3 remain understated by an unknown amount. They should be read as exploratory.

(An earlier version of this table reported `hives_in_days` as the complement of `hives_in_seconds`, which silently counted every undetermined hive as day-scaled and made the columns sum to more than 78. The counts above are computed by testing for day scale directly.)

```
In [8]: proximity = show("extremes_event_proximity.csv")

# The published percentages divide by the rows that carry a logged event of that type,
# not by every row in the group. Logging coverage is itself different between the two
# groups -- 30.6% of extremes carry a logged treatment event against 44.1% of routine
# rows -- so that denominator selects as well as measures. Recompute against all rows.
proximity["within_1_day_n"] = (
    proximity.pct_within_1_day * proximity.n_with_event_logged / 100
).round().astype(int)
proximity["within_1_day_pct_of_all"] = (
    100 * proximity.within_1_day_n / proximity.n_rows
).round(2)

comparison = proximity.pivot(
    index="event", columns="group",
    values=["pct_with_event_logged", "pct_within_1_day",
            "within_1_day_pct_of_all", "within_1_day_n"],
)
display(comparison)
```

event	pct_with_event_logged		pct_within_1_day		within_1_day_pct_of_all		within_1_day_n	
	group		group		group		group	
	extreme	routine	extreme	routine	extreme	routine	extreme	routine
died	21.57	15.29	1.04	1.85	0.22	0.28	1.0	73.0
feeding	59.55	61.02	13.21	8.69	7.87	5.30	35.0	1367.0
honey	60.45	49.37	7.81	8.68	4.72	4.28	21.0	1104.0
queencell	37.98	34.17	10.65	9.97	4.04	3.41	18.0	878.0
swarming	28.99	29.11	1.55	4.07	0.45	1.18	2.0	305.0
treatment	30.56	44.09	10.29	7.41	3.15	3.27	14.0	842.0

The table above reports the same comparison under two denominators, and they do not agree.

`pct_within_1_day` is the published figure: the share of rows that carry a logged event of that type falling within one day of it. That denominator is not neutral, because logging coverage differs between the groups — 30.6% of extreme observations carry a logged treatment event against 44.1% of routine observations. Conditioning on the presence of a log therefore compares two differently selected populations. `within_1_day_pct_of_all` divides by every row in the group instead, which is the quantity the interpretation below actually needs.

Under the all-rows denominator:

- **Feeding** remains directionally higher among extremes: 7.9% of extreme observations lie within one day of a logged feeding event against 5.3% of routine observations. The difference rests on 35 extreme rows, and across the twelve event-by-window comparisons made here it would not survive a correction for multiple testing.
- **Treatment does not survive.** The conditional figures (10.3% against 7.4%) reverse to 3.15% against 3.27% — extreme observations are, if anything, marginally less likely to sit within a day of a logged treatment event. The apparent association was produced by the difference in logging coverage, not by event timing.

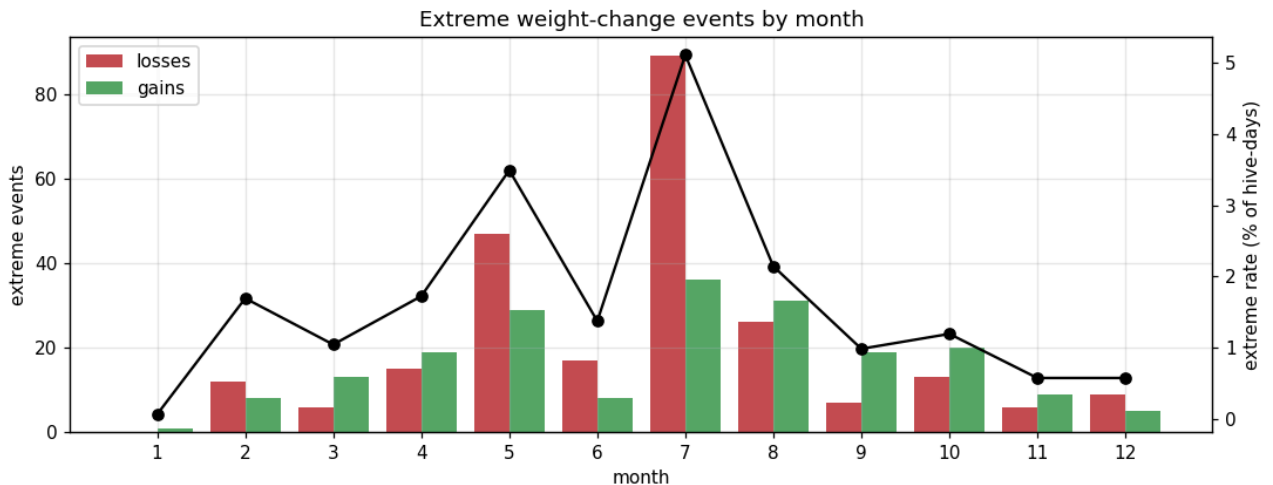
- **Died (1.04%) and swarming (1.55%) rest on one and two extreme observations respectively.** Those percentages should not be read as estimates of anything.

Two further limitations bound all of these figures. First, the unit normalisation above resolves only a minority of hives — the undetermined group is the largest for every column — and unresolved hives are left unconverted, so any that are seconds-scaled still cannot register as near an event. Second, beekeeper logs are self-entered and incomplete.

The conclusion Milestone 3 reached — that extreme weight changes are not meaningfully closer to logged beekeeper events than routine days — is largely unchanged by this analysis. Feeding is the one category with a difference in the expected direction, it is small, and it is not robust enough to carry weight on its own. Event proximity does not explain the extreme-event population.

3.4 When do extremes happen?

```
In [9]: monthly = show("extremes_by_month.csv")
fig, ax = plt.subplots(figsize=(10, 4))
ax.bar(monthly.month - 0.2, monthly.extreme_losses, width=0.4, label="losses", color="#c44e52")
ax.bar(monthly.month + 0.2, monthly.extreme_gains, width=0.4, label="gains", color="#55a868")
ax2 = ax.twinx(); ax2.plot(monthly.month, monthly.extreme_rate_pct, color="black", marker="o", label="rate")
ax2.set_ylabel("extreme rate (% of hive-days)"); ax2.grid(False)
ax.set_xlabel("month", ylabel="extreme events", title="Extreme weight-change events by month",
            xticks=range(1, 13))
ax.legend(loc="upper left"); plt.tight_layout(); plt.show()
display(monthly)
```



	month	hive_days	extremes	extreme_losses	extreme_gains	extreme_rate_pct
0	1	1624	1	0	1	0.06
1	2	1186	20	12	8	1.69
2	3	1828	19	6	13	1.04
3	4	1978	34	15	19	1.72
4	5	2179	76	47	29	3.49
5	6	1826	25	17	8	1.37
6	7	2447	125	89	36	5.11
7	8	2659	57	26	31	2.14
8	9	2652	26	7	19	0.98
9	10	2765	33	13	20	1.19
10	11	2615	15	6	9	0.57
11	12	2456	14	9	5	0.57

The extreme-event analysis reveals a pronounced seasonal pattern. Extreme observations occur most frequently during the warmer months, with July exhibiting the highest extreme-event rate (5.11%), followed by May (3.49%), while January

shows a substantially lower rate (0.06%). July also contains considerably more extreme losses than gains (89 versus 36).

The concentration of extreme weight losses during the summer months suggests that these observations may reflect processes with a seasonal component. Potential explanations include beekeeper management activities such as honey harvesting, seasonal changes in colony dynamics, environmental conditions, or combinations of these factors. Because the available data do not independently verify the cause of each extreme observation, the seasonal pattern should be interpreted as evidence for further investigation rather than attribution to a specific mechanism.

This finding expands the Milestone 3 analysis by demonstrating that extreme observations are not distributed uniformly throughout the year. Incorporating seasonality into subsequent modeling and event analysis may therefore improve characterization of these relatively rare observations.

3.5 Sensitivity of extreme-event identification to weight-series selection

```
In [10]: display(show("extremes_weight_series.csv"))
```

	modelling_rows	extremes_weight_kg	extremes_weight_kg_noOutlier	extremes_removed_by_cleaning	target_correlation
0	26215	445.0	101.0	419.0	0.1243

The published dataset provides both the original weight_kg series and a corresponding weight_kg_noOutlier series in which abrupt changes identified by the publisher's rate-based filtering procedure have been adjusted. Applying the same daily modeling-grain construction to the two series produces substantially different extreme-event populations: 445 extreme observations are identified using weight_kg, compared with 101 when using weight_kg_noOutlier. Of the original extreme observations, 419 are affected by the publisher's filtering procedure.

This difference demonstrates that extreme-event identification is highly sensitive to the choice of weight series. The publisher documentation indicates that abrupt weight changes exceeding the rate threshold are frequently associated with beekeeper activity; however, the filtering procedure does not establish the cause of each individual extreme observation. The large reduction in extreme observations therefore provides additional evidence that management-related or measurement-related abrupt changes may contribute substantially to the extreme-event population, but it does not permit direct causal attribution.

The project retains weight_kg for the primary target to maintain continuity with the modeling framework established in Milestone 3 and to avoid automatically removing potentially meaningful biological or management-related changes. The alternative series is instead used as a sensitivity analysis. Comparing results across the two representations helps quantify how publisher-defined filtering decisions influence the target distribution and subsequent model performance.

3.6 Threshold sensitivity

```
In [11]: display(show("threshold_sweep.csv"))
```

	definition	extremes	extreme_pct	routine_rows	routine_mae_oracle_gated	routine_r2_oracle_gated	classifier_pr_auc	cla
0	absolute 3 kg	834	3.181	25381	0.2387	0.2510	0.3154	
1	absolute 5 kg	445	1.698	25770	0.3005	0.2274	0.2276	
2	absolute 7 kg	281	1.072	25934	0.3303	0.2107	0.1833	
3	absolute 10 kg	178	0.679	26037	0.3565	0.2009	0.1435	
4	hive-relative 5 MAD	2873	10.959	23342	0.1425	0.1691	0.5230	

The ±5 kg threshold established in Milestone 3 provided a consistent operational definition of an extreme weight change. Because this absolute threshold applies equally across hives of substantially different sizes, Milestone 4 evaluates the sensitivity of the results to alternative thresholds and a hive-relative definition.

The threshold sensitivity analysis demonstrates a consistent trade-off between event frequency and classification performance. Lower absolute thresholds identify more positive observations and produce higher classifier PR-AUC, while higher thresholds define increasingly rare events. The hive-relative definition (>5 MAD from each hive's own weight-change distribution) identifies 10.96% of hive-days and achieves the highest PR-AUC (0.523), suggesting that a hive-specific definition may better capture unusual changes relative to each colony's typical variability.

The ± 5 kg threshold is retained in the current analysis for continuity and comparability with Milestone 3. However, the performance of the hive-relative definition suggests that adaptive thresholds may be valuable for future development of an operational early-warning system.

4. Validation Framework

Milestone 4 expands the validation framework established in Milestone 3 to provide a more robust assessment of model generalization. The updated evaluation incorporates date-based chronological splitting, rolling-origin cross-validation, repeated model evaluation where appropriate, comparison with naive baselines, and performance assessment on the complete test distribution. These additions allow model performance to be evaluated across multiple time periods while preserving the temporal structure of the data.

4.1 Date-based chronological splitting

The chronological train-test split was refined for Milestone 4 to operate on measurement dates rather than row positions. Because multiple hives contribute observations on the same calendar date, a row-index split can place observations from a single date in both the training and test sets. Although the observations represent different hives, separating the datasets at a date boundary provides a clearer temporal distinction between model development and evaluation.

The updated `date_chronological_split` function therefore partitions observations using the measurement date itself. An integrity check verifies that the latest training date precedes the earliest testing date (`max(train_date) < min(test_date)`), ensuring that the evaluation period occurs entirely after the training period.

```
In [12]: folds = ev.rolling_origin_cv(model_df, n_splits=4, horizon_months=6, min_train_months=12)
display(pd.DataFrame([f.describe() for f in folds]))

for fold in folds: # the assertion each fold already passed at construction
    ev.assert_split_integrity(model_df, fold)
print("split integrity: every fold's training data ends strictly before its test data begins")
```

	fold	n_train	n_test	train_end	test_start	test_end
0	fold_1_2020-12-25	9278	4043	2020-12-24	2020-12-25	2021-06-24
1	fold_2_2021-06-25	13321	4525	2021-06-24	2021-06-25	2021-12-24
2	fold_3_2021-12-25	17846	4376	2021-12-24	2021-12-25	2022-06-19
3	fold_4_2022-06-25	22222	3867	2022-06-19	2022-07-05	2022-12-24

split integrity: every fold's training data ends strictly before its test data begins

4.2 Rolling-origin cross-validation

To evaluate model performance across multiple time periods, Milestone 4 uses rolling-origin cross-validation with four expanding-window folds. For each fold, the training set includes all observations available up to the evaluation boundary, and the subsequent six months are reserved for testing. This approach preserves the temporal ordering of the data while allowing model performance to be assessed across different seasonal periods. Compared with relying on a single chronological split, rolling-origin validation provides a more robust estimate of model generalization and helps identify variation in predictive performance over time.

4.3 Naive baselines

Naive baseline models provide reference points for determining whether the predictive models improve meaningfully upon simple forecasting strategies. Because the target has an overall mean near zero (0.002 kg), several baseline approaches were evaluated across the same four rolling-origin folds, including prediction of zero, the training-set mean, hive-specific means, hive-month means, and persistence based on the previous day's change.

```
In [13]: baselines = show("baselines.csv")
display(ev.fold_summary(baselines).round(4))
```

	model	folds	mae_mean	mae_sd	rmse_mean	rmse_sd	r2_mean	r2_sd	n
0	predict_zero	4	0.5128	0.0782	1.7425	0.2017	-0.0029	0.0028	16811
1	train_mean	4	0.5157	0.0753	1.7431	0.2020	-0.0036	0.0034	16811
2	hive_mean	4	0.5466	0.0817	1.7504	0.2037	-0.0119	0.0075	16811
3	hive_month_mean	4	0.5862	0.0848	1.7916	0.2223	-0.0594	0.0377	16811
4	persistence	4	0.5974	0.0569	2.1681	0.1983	-0.5825	0.3263	16811

Among the evaluated baselines, predicting zero produced the lowest mean MAE (0.513 kg), making it the strongest naive benchmark for subsequent model comparisons. The persistence baseline, which predicts the next change using the previous day's change, produced a higher mean MAE (0.597 kg) and negative mean R^2 (-0.583). This indicates that simply carrying the previous day's weight change forward does not provide an effective prediction strategy at the daily grain.

5. Models

All candidate models are evaluated using the same rolling-origin cross-validation framework described above. In addition to MAE, RMSE, and R^2 , model performance is summarized using a skill score representing the percentage reduction in MAE relative to the strongest naive baseline. Positive skill scores indicate improvement over the baseline, while negative values indicate that the model did not outperform the reference prediction strategy. Using a common validation framework and baseline allows performance differences among candidate models to be compared consistently.

5.1 The model board

```
In [14]: summary = show("model_summary.csv")
display(summary.round(4))
```

	model	folds	mae_mean	mae_sd	rmse_mean	rmse_sd	r2_mean	r2_sd	n	skill_mean	skill_sd
0	predict_zero	4	0.5128	0.0782	1.7425	0.2017	-0.0029	0.0028	16811	NaN	NaN
1	train_mean	4	0.5157	0.0753	1.7431	0.2020	-0.0036	0.0034	16811	NaN	NaN
2	history / HistGB	4	0.5423	0.0777	1.6980	0.1338	0.0359	0.1301	16811	-0.0614	0.0894
3	hive_mean	4	0.5466	0.0817	1.7504	0.2037	-0.0119	0.0075	16811	NaN	NaN
4	history+sensors / HistGB	4	0.5667	0.0831	1.7042	0.1526	0.0305	0.1238	16811	-0.1105	0.1221
5	hive_month_mean	4	0.5862	0.0848	1.7916	0.2223	-0.0594	0.0377	16811	NaN	NaN
6	persistence	4	0.5974	0.0569	2.1681	0.1983	-0.5825	0.3263	16811	NaN	NaN
7	history+sensors / RF	4	0.6695	0.0949	2.0783	0.2687	-0.4699	0.4108	16811	-0.3117	0.1309
8	M3 features / GB	4	0.7833	0.1981	2.5111	0.5713	-1.1567	0.7625	16811	-0.5256	0.3090
9	M3 features / RF	4	0.9769	0.1504	2.4982	0.5066	-1.1264	0.7115	16811	-0.9136	0.2082

Reading the board. `mae_mean`, `rmse_mean`, `r2_mean` and `skill_mean` are unweighted means across the four folds, and `skill_mean` is the mean of the four per-fold skill scores rather than a skill score computed from the mean MAE — the two differ slightly because a mean of ratios is not the ratio of means. The `n` column is the *sum* of test rows across folds, not the denominator of those means; fold test sizes range from 3,867 to 4,525, so the unweighted mean is close to but not identical to a pooled figure.

The rolling-origin evaluation demonstrates that model performance varies substantially across feature sets and algorithms. Among the learned single-stage models, the history-only HistGradientBoosting model produced the strongest overall performance, with a mean MAE of 0.542 kg, RMSE of 1.698 kg, and R^2 of 0.036 across the four validation folds. Although its MAE did not improve upon the predict-zero baseline (MAE = 0.513 kg), it achieved a lower RMSE than the baseline (1.698 kg compared with 1.743 kg) and a small positive mean R^2 .

The difference between the MAE and RMSE results provides additional insight into model behavior. The near-zero target distribution makes prediction of zero a competitive strategy for minimizing absolute error across routine observations. In contrast, RMSE places greater weight on larger prediction errors, and the lower RMSE of the history-only HistGradientBoosting model suggests that it provides some predictive value for larger deviations even though it does not improve overall MAE.

Models using the original Milestone 3 feature configuration produced weaker performance under the expanded rolling-origin validation framework, with mean MAE values of 0.783 kg for Gradient Boosting and 0.977 kg for Random Forest and negative mean R^2 values. These results should not be interpreted as invalidating the earlier analysis. Rather, the expanded evaluation demonstrates that performance estimates are sensitive to the validation strategy, preprocessing decisions, and evaluation population. The Milestone 4 framework therefore provides a more conservative assessment of how well the models generalize across multiple future time periods.

5.2 Final-fold model evaluation

Which feature set the rest of the notebook uses. Two HistGradientBoosting configurations appear in the board above: `history`, which uses weight-history features only (fold-mean MAE 0.542 kg), and `history+sensors`, which adds the hive sensor channels (0.567 kg). The history-only configuration has the lower fold-mean MAE. Every downstream analysis in Sections 5.3 through 6.1, however — the framing comparison, the extreme-event classifier, seed stability, permutation importance, and the segmented report — is built on `history+sensors`, because those analyses have to include the sensor channels in order to say anything about them.

The refit below therefore also uses `history+sensors`, so that one model stands behind one narrative. An earlier draft of this notebook refit the history-only configuration here while the seasonal commentary in Section 6.1 was written from the `history+sensors` run, which produced a table that disagreed with the paragraph beneath it. Choosing the sensor configuration for this cell costs about 0.011 kg of MAE on the final fold and removes the inconsistency.

The model comparison above is generated through `scripts/run_evaluation.py`. Refitting the same configuration here provides a reproducible example of model training and evaluation under the preprocessing and temporal-validation framework applied throughout the analysis.

```
In [15]: # `history+sensors` -- the configuration carried through Sections 5.3-6.1. See the note
# above for why this is not the history-only model that tops the fold-mean MAE board.
matrix = features.build_feature_matrix(model_df, feature_set="history+sensors", impute="none")
fold = folds[-1]
X_train, y_train = matrix.X.iloc[fold.train_index], matrix.y.iloc[fold.train_index]
X_test, y_test = matrix.X.iloc[fold.test_index], matrix.y.iloc[fold.test_index]

winner = models.make_regressor("hist_gb", seed=42).fit(X_train, y_train)
predictions = winner.predict(X_test)

metrics = ev.regression_metrics(y_test, predictions)
naive_mae = ev.best_baseline_mae(baselines, fold.name)
print(f"feature set : history+sensors ({matrix.X.shape[1]} columns)")
print(f"fold : {fold.name} ({fold.test_start.date()} to {fold.test_end.date()}, n={metrics['n']})")
print(f"MAE : {metrics['mae']:.4f} kg (best naive: {naive_mae:.4f})")
print(f"RMSE : {metrics['rmse']:.4f} kg")
print(f"R2 : {metrics['r2']:.4f}")
print(f"skill vs naive: {ev.skill_score(metrics['mae'], naive_mae):+.1%}")

feature set : history+sensors (20 columns)
fold : fold_4_2022-06-25 (2022-07-05 to 2022-12-24, n=3,867)
MAE : 0.4510 kg (best naive: 0.4536)
RMSE : 1.6203 kg
R2 : 0.1630
skill vs naive: +0.6%
```

On reproducing this cell. The fit is seeded (`seed=42`) and deterministic for a fixed library stack: repeated runs in one environment return identical metrics. It is not stable *across* library versions, because `HistGradientBoostingRegressor` enables early stopping automatically above 10,000 training rows and the binning and validation-split behaviour behind it has changed between scikit-learn releases. An earlier execution of this notebook stored MAE 0.4579 and a skill score of -1.0% for the history-only configuration; re-running that same code under the versions printed in Section 1 returns 0.4404 and +2.9%, which changes the sign of the headline. The outputs stored here were produced by the stack named in Section 1, and that line should be checked first if a re-run disagrees.

This is also the reason the tables read from `results/` are regenerated by `scripts/run_evaluation.py` rather than recomputed inline: it keeps every model number in the write-up traceable to a single run. The consequence is that those tables are only as current as the last time that script was executed.

5.3 Comparison of modeling frameworks

```
In [16]: framings = show("framings.csv")
table = (framings.groupby(["framing", "honest_label"])["mae", "rmse", "r2", "skill_vs_naive"]
        .mean().round(4).reset_index().sort_values("mae"))
display(table)
```

	framing	honest_label	mae	rmse	r2	skill_vs_naive
0	routine only, oracle-gated	UPPER BOUND - test rows selected using the label	0.3151	0.6500	0.1544	0.3797
3	two-stage, unfiltered test (hard gate)	deployable	0.4794	1.7016	0.0435	0.0586
2	two-stage, unfiltered test (blended)	deployable	0.5157	1.6653	0.0778	-0.0119
1	single regressor, all data	realistic	0.5667	1.7042	0.0305	-0.1105

```
In [17]: # Does the gate earn its place? The two-stage model differs from the single regressor in
# two ways at once -- it trains its routine regressor on routine rows only, AND it routes
# with a classifier -- and only the second is the two-stage idea. This control keeps the
# training filter and drops the gate: one regressor, fitted on training rows with
# |change| <= 5 kg, applied to every test row.
control_rows = []
for cv_fold in folds:
    Xa, ya = matrix.X.iloc[cv_fold.train_index], matrix.y.iloc[cv_fold.train_index]
    Xb, yb = matrix.X.iloc[cv_fold.test_index], matrix.y.iloc[cv_fold.test_index]
    routine_train = (ya.abs() <= 5.0).to_numpy()
    control = models.make_regressor("hist_gb", seed=42).fit(Xa[routine_train], ya[routine_train])
    control_metrics = ev.regression_metrics(yb, control.predict(Xb))
    control_rows.append({
        "fold": cv_fold.name, **control_metrics,
        "skill_vs_naive": ev.skill_score(control_metrics["mae"],
                                         ev.best_baseline_mae(baselines, cv_fold.name)),
    })
control_board = pd.DataFrame(control_rows)

ablation = pd.concat([
    table[["framing", "honest_label", "mae", "rmse", "r2", "skill_vs_naive"]],
    pd.DataFrame([
        {
            "framing": "routine-train regressor, no gate (control)",
            "honest_label": "deployable",
            **control_board[["mae", "rmse", "r2", "skill_vs_naive"]].mean().to_dict(),
        }
    ]),
], ignore_index=True).sort_values("mae").round(4)
display(ablation)
```

	gate = table.loc[table.framing.str.contains("hard gate"), "skill_vs_naive"].iloc[0]					
	print(f"two-stage hard gate : {gate:+.1%} skill")					
	print(f"control, no gate : {control_board.skill_vs_naive.mean():+.1%} skill "					
	f"(per fold {' '.join(f'{s:+.1%}' for s in control_board.skill_vs_naive)}))"					
	framing	honest_label	mae	rmse	r2	skill_vs_naive
0	routine only, oracle-gated	UPPER BOUND - test rows selected using the label	0.3151	0.6500	0.1544	0.3797
4	routine-train regressor, no gate (control)	deployable	0.4778	1.6932	0.0522	0.0616
1	two-stage, unfiltered test (hard gate)	deployable	0.4794	1.7016	0.0435	0.0586
2	two-stage, unfiltered test (blended)	deployable	0.5157	1.6653	0.0778	-0.0119
3	single regressor, all data	realistic	0.5667	1.7042	0.0305	-0.1105

two-stage hard gate : +5.9% skill
control, no gate : +6.2% skill (per fold -0.2%, +5.8%, +13.7%, +5.3%)

To evaluate the influence of extreme observations on model performance, four modeling frameworks were compared using the rolling-origin validation procedure. These frameworks distinguish between evaluation restricted to routine observations, two-stage approaches that first classify potential extreme events, and a single-regressor approach evaluated across the complete target distribution.

The routine-only framework produced the lowest MAE (0.315 kg) and highest R^2 (0.154); however, this result should be interpreted as an upper-bound estimate because routine test observations are identified using the observed target value. In a prospective prediction setting, the extreme status of the next-day observation would not be known in advance. Therefore, this framework is useful for characterizing predictability within routine observations but does not represent deployable performance.

The two-stage hard-gated framework addresses this limitation by first classifying each observation as routine or extreme and then routing the observation to the corresponding regression model. When evaluated on the complete, unfiltered test distribution, this approach achieved an MAE of 0.479 kg, RMSE of 1.702 kg, and R^2 of 0.044 — a 5.9% improvement in MAE relative to the strongest naive baseline, and the best deployable MAE among the four framings originally compared.

What that 5.9% is actually attributable to. The two-stage model differs from the single regressor in two ways at once, and only one of them is the two-stage idea. It trains its routine regressor on routine training rows only, *and* it routes predictions with a classifier. The ablation cell above isolates the two by keeping the training filter and discarding the gate: a single HistGradientBoosting regressor fitted on training rows with $|\text{change}| \leq 5$ kg, applied to every test row with no classifier involved.

That control reaches an MAE of 0.478 kg and a 6.2% skill score — it matches, and very slightly exceeds, the two-stage model's 0.479 kg and 5.9%. The improvement over the single regressor is therefore attributable to excluding extreme observations from the *training* set, not to gating predictions at inference time. Because training labels are known when the model is fitted, the control is equally deployable, and it is the simpler of the two.

This does not make the classifier useless — Section 5.4 shows it carries real signal about which days will be extreme — but it does mean that routing on that signal did not improve next-day regression error here. The blended variant, which weights the two regressors by predicted probability, achieved a higher R^2 (0.078) but did not improve on the naive baseline in MAE, and the single regressor trained on all data produced the weakest deployable performance. Taken together, the framings show that the practical gain available at the daily grain comes from keeping extreme observations out of the regressor's training distribution, and that it is modest.

5.4 The extreme-event classifier

```
In [18]: classifier = show("classifier.csv")
display(classifier.round(4))
print(f"mean PR-AUC {classifier.pr_auc.mean():.3f} against a no-skill rate of "
      f"{classifier.pr_auc_no_skill.mean():.3f} -> {classifier.pr_auc_lift.mean():.1f}x lift")
```

	fold	pr_auc	roc_auc	positive_rate	pr_auc_no_skill	pr_auc_lift
0	fold_1_2020-12-25	0.1656	0.8059	0.0096	0.0096	17.1637
1	fold_2_2021-06-25	0.1661	0.8537	0.0170	0.0170	9.7585
2	fold_3_2021-12-25	0.1783	0.7986	0.0169	0.0169	10.5467
3	fold_4_2022-06-25	0.3097	0.8529	0.0184	0.0184	16.8666

mean PR-AUC 0.205 against a no-skill rate of 0.015 -> 13.6x lift

Extreme weight changes represent approximately 1.7% of the daily observations, creating a substantial class imbalance. Under this distribution, overall classification accuracy would provide limited information because a model predicting every observation as routine would achieve approximately 98.3% accuracy. Precision-recall area under the curve (PR-AUC) is therefore emphasized because it evaluates the classifier's ability to identify the relatively rare extreme observations. Performance is compared with a no-skill PR-AUC equal to the positive-class prevalence within each validation fold.

Across the four rolling-origin validation folds, PR-AUC ranged from 0.166 to 0.310, compared with no-skill baselines ranging from 0.010 to 0.018. The mean PR-AUC was 0.205, approximately 13.6 times the mean no-skill rate. ROC-AUC ranged from approximately 0.80 to 0.85 across the folds. These results indicate that the available predictors contain

information useful for distinguishing observations associated with extreme next-day weight changes. However, classification performance varies across validation periods, and these results do not establish the underlying cause of the extreme events. The classifier is therefore best interpreted as identifying elevated risk of an extreme next-day observation rather than predicting a specific biological or beekeeper-related event.

5.5 Model Stability and Generalization

```
In [19]: stability = show("seed_stability.csv")
display(stability.round(4))
rf = stability[stability.model == "rf"]; hgb = stability[stability.model == "hist_gb"]
print(f"RF      MAE range: {rf.mae.min():.4f} - {rf.mae.max():.4f}")
print(f"HistGB  MAE range: {hgb.mae.min():.4f} - {hgb.mae.max():.4f}")
print(f"non-overlapping ranges: {ev.ranking_is_significant(rf, hgb)}")
```

	seed	mae	rmse	r2	bias	n	model
0	0	0.5059	1.6928	0.0864	0.0818	3867	rf
1	1	0.5068	1.6931	0.0861	0.0845	3867	rf
2	2	0.5029	1.6850	0.0949	0.0806	3867	rf
3	0	0.4339	1.6414	0.1411	0.0870	3867	hist_gb
4	1	0.4568	1.6200	0.1633	0.0481	3867	hist_gb
5	2	0.4461	1.6314	0.1516	0.0640	3867	hist_gb

```
RF      MAE range: 0.5029 - 0.5068
HistGB  MAE range: 0.4339 - 0.4568
non-overlapping ranges: True
```

Model stability was evaluated across multiple random seeds to determine whether the relative performance of Random Forest and HistGradientBoosting depended strongly on stochastic variation during model fitting. Both models were fitted on the `history+sensors` feature matrix and evaluated on the final rolling-origin fold, matching the configuration used in Sections 5.2 through 6.1.

Across the evaluated seeds, Random Forest produced MAE values ranging from 0.503 to 0.507 kg, while HistGradientBoosting produced MAE values ranging from 0.434 to 0.457 kg. The observed MAE ranges did not overlap, indicating that the relative ranking of these models was stable across the tested seeds. This comparison is intended as a robustness assessment rather than a formal statistical significance test, and three seeds bound the spread loosely: the range reported here should be read as a lower bound on seed-to-seed variation rather than its full extent.

```
In [20]: hive_cv = show("hive_generalisation.csv")

# `evaluate_folds` was called for these folds without a baseline board, so the published
# table ships no bar to read the MAE against. Attach one: predict_zero's MAE on a block of
# held-out hives is just the mean |target| over those rows, which needs no model.
hive_folds = ev.grouped_hive_cv(model_df, n_folds=5)
hive_cv["predict_zero_mae"] = [
    float(model_df.iloc[f.test_index][data.TARGET].abs().mean()) for f in hive_folds
]
hive_cv["skill_vs_zero"] = 1 - hive_cv.mae / hive_cv.predict_zero_mae
display(hive_cv.round(4))

print(f"leave-hives-out MAE {hive_cv.mae.mean():.3f} +/- {hive_cv.mae.std():.3f}, "
      f"R2 {hive_cv.r2.mean():.3f} +/- {hive_cv.r2.std():.3f}")
print(f"skill vs predict_zero: mean {hive_cv.skill_vs_zero.mean():+.1%}, "
      f"per fold {hive_cv.skill_vs_zero.min():+.1%} to {hive_cv.skill_vs_zero.max():+.1%}, "
      f"negative in {(hive_cv.skill_vs_zero < 0).sum()} of {len(hive_cv)} folds")
```

	fold	model	mae	rmse	r2	bias	n	predict_zero_mae	skill_vs_zero
0	hives_1	hist_gb (leave-hives-out)	0.6633	2.1956	0.0493	-0.0081	4971	0.6247	-0.0617
1	hives_2	hist_gb (leave-hives-out)	0.4432	1.5549	0.0943	0.0131	4709	0.4688	0.0545
2	hives_3	hist_gb (leave-hives-out)	0.4954	1.4949	0.0936	-0.0371	7802	0.4864	-0.0187
3	hives_4	hist_gb (leave-hives-out)	0.6038	2.1074	0.1347	-0.0251	4356	0.6365	0.0514
4	hives_5	hist_gb (leave-hives-out)	0.4220	1.4052	0.0831	0.0057	4377	0.4284	0.0150

leave-hives-out MAE 0.526 +/- 0.104, R2 0.091 +/- 0.031
skill vs predict_zero: mean +0.8%, per fold -6.2% to +5.5%, negative in 2 of 5 folds

In addition to evaluating stability across random seeds, model behaviour on unseen colonies was assessed using a leave-hives-out validation strategy, which evaluates performance on hives excluded from model training.

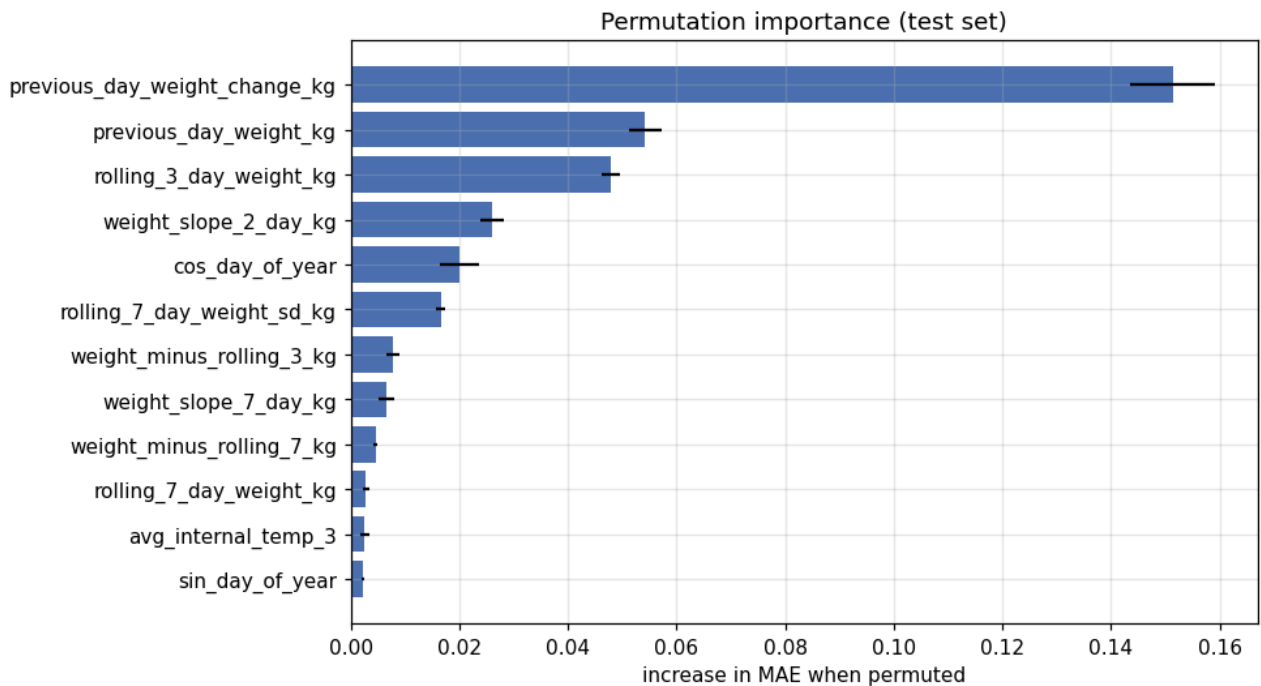
Two properties of this split limit what it can establish. It partitions on `hive_id` only, so training and test rows span the same 2019–2022 window and the model is fitted on calendar dates later than many of the rows it is scored on. It is therefore a test of transfer across colonies, not across time, and it is a weaker temporal test than the rolling-origin folds rather than a stricter one overall. It is also reported without a reference prediction in the published table, which the cell above supplies.

Across the five folds, MAE ranged from approximately 0.42 to 0.66 kg and R^2 from approximately 0.05 to 0.13. Measured against a predict-zero baseline computed within each held-out block, mean skill is +0.8%, ranging from -6.2% to +5.5%, and is negative in two of the five folds. The model is therefore approximately level with predicting no change on colonies it was not trained on, with the sign of the difference varying by fold.

These results are consistent with the model retaining some capability on unseen hives, but they do not establish it: a mean skill score of under one percent with inconsistent sign is not distinguishable from no transfer at this sample size. They also represent internal validation within a single study population and should not be read as evidence of generalizability to hives, geographic regions, sensor systems, or management conditions outside the dataset.

5.6 Permutation Feature Importance

```
In [21]: importance = show("permutation_importance.csv").head(12)
fig, ax = plt.subplots(figsize=(9, 5))
ax.barh(importance.feature[:-1], importance.importance_mean[:-1],
        xerr=importance.importance_sd[:-1], color="#4c72b0")
ax.set(xlabel="increase in MAE when permuted", title="Permutation importance (test set)")
plt.tight_layout(); plt.show()
display(importance.round(5))
```



	feature	importance_mean	importance_sd
0	previous_day_weight_change_kg	0.15133	0.00775
1	previous_day_weight_kg	0.05418	0.00296
2	rolling_3_day_weight_kg	0.04788	0.00158
3	weight_slope_2_day_kg	0.02601	0.00216
4	cos_day_of_year	0.02003	0.00361
5	rolling_7_day_weight_sd_kg	0.01655	0.00092
6	weight_minus_rolling_3_kg	0.00775	0.00109
7	weight_slope_7_day_kg	0.00650	0.00144
8	weight_minus_rolling_7_kg	0.00452	0.00034
9	rolling_7_day_weight_kg	0.00280	0.00065
10	avg_internal_temp_3	0.00254	0.00081
11	sin_day_of_year	0.00226	0.00023

Permutation feature importance was used to evaluate the contribution of individual predictors to model performance on held-out data. The values below are computed for the `history+sensors` HistGradientBoosting model on the final rolling-origin fold. Unlike impurity-based feature importance, permutation importance measures the change in predictive error that occurs when the values of a feature are randomly permuted, providing an estimate of the model's dependence on that feature for prediction.

`previous_day_weight_change_kg` produced the largest increase in MAE when permuted, substantially exceeding the importance of the remaining predictors. Previous-day hive weight and the three-day rolling weight were the next most influential variables, while several additional weight-history features provided smaller contributions. In contrast, the internal-temperature and seasonal variables shown here contributed relatively little to predictive performance. Overall, these results indicate that recent weight dynamics provide the strongest predictive information within the evaluated feature set.

These importance values should be interpreted cautiously because several weight-history variables are strongly related by construction. When correlated predictors contain overlapping information, permutation of a single feature may underestimate its contribution because related variables can partially compensate for the disrupted predictor. The results therefore provide stronger evidence about the importance of recent weight history as a group than about the precise ranking of individual weight-derived features.

6. Results and Interpretation

6.1 Model Performance Across Data Segments

Model performance was further evaluated across temporal and target-distribution segments to determine whether predictive accuracy remained consistent throughout the test period. The segmentation below uses the `history+sensors` model refit in Section 5.2, evaluated on the final rolling-origin fold. Because model skill is measured relative to a naive reference prediction, interpretation of segment-level skill depends on how the comparison baseline is defined.

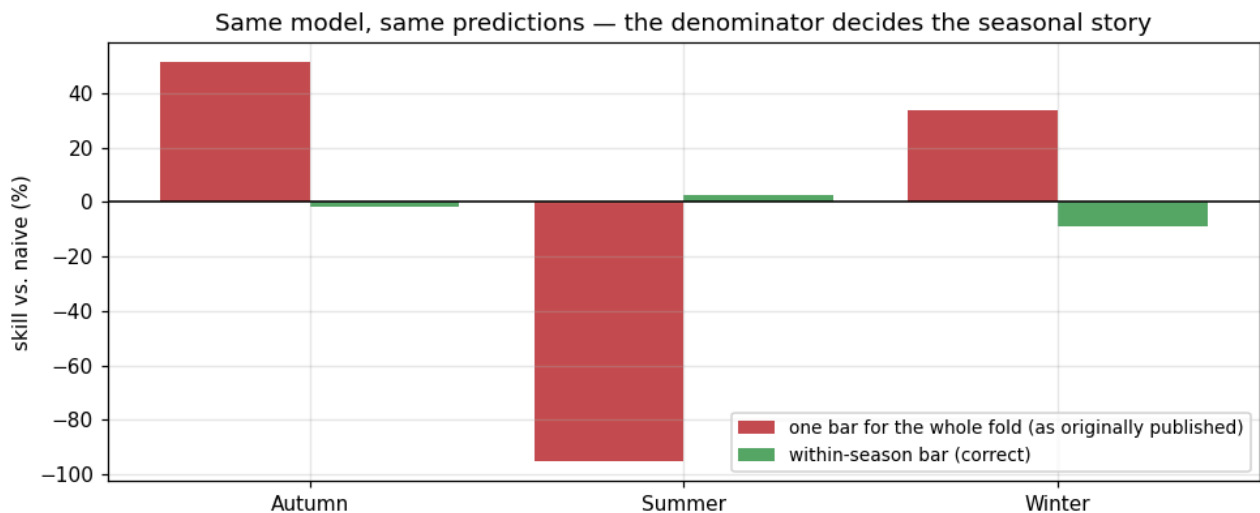
For segments that are known at prediction time, such as season or calendar month, performance can be compared with a naive baseline calculated within the same segment. This provides a context-specific assessment of whether the model improves upon a simple prediction strategy under the conditions represented by that segment. In contrast, segments defined using the observed target, such as weight-change magnitude, are not known before prediction. Evaluating those groups against segment-specific baselines would therefore incorporate information unavailable at prediction time. For these analyses, the pooled naive baseline from the complete test fold provides the more appropriate reference.

Accordingly, the following analyses report segment-level performance using the baseline definition appropriate to the information that would be available at prediction time. This distinction allows variation in model performance across seasons, months, and weight-change magnitudes to be examined without introducing target-dependent information into the evaluation framework.

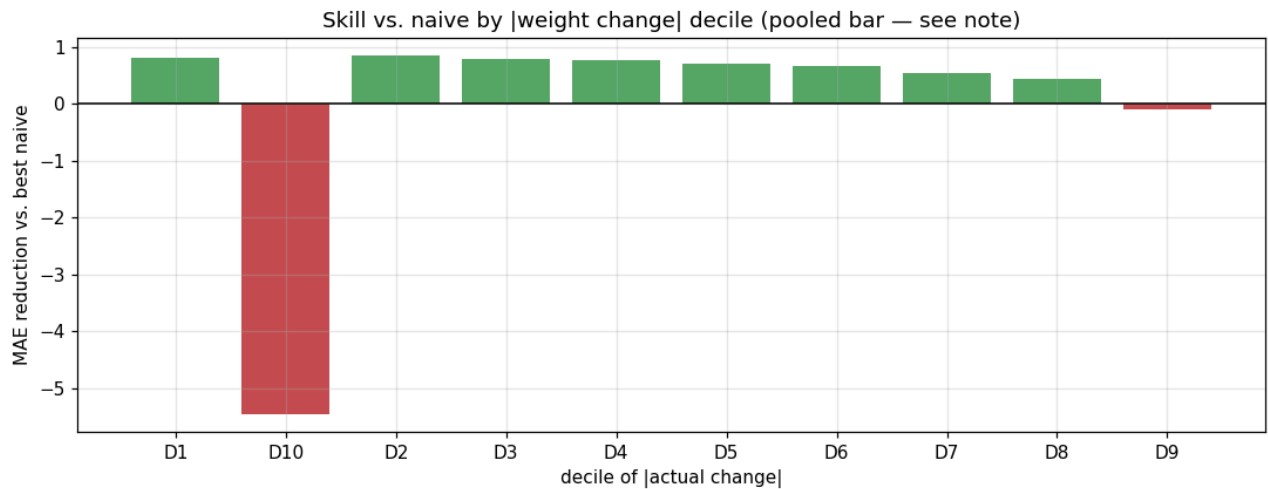
```
In [22]: segmented = ev.segmented_report(
    y_test, predictions, matrix.meta.iloc[fold.test_index],
    by=("season", "month", "weight_change_decile"),
    baseline_mae=naive_mae,
    baseline_predictions=ev.naive_baselines(model_df, fold),
)
seasons = segmented[segmented.segment == "season"]
display(seasons[["value", "n", "mae", "baseline_mae", "baseline_rule",
    "skill_vs_naive", "skill_vs_pooled_naive"]].round(4))

fig, ax = plt.subplots(figsize=(9, 3.8))
x = np.arange(len(seasons))
ax.bar(x - 0.2, 100 * seasons.skill_vs_pooled_naive, 0.4,
    color="#c44e52", label="one bar for the whole fold (as originally published)")
ax.bar(x + 0.2, 100 * seasons.skill_vs_naive, 0.4,
    color="#55a868", label="within-season bar (correct)")
ax.axhline(0, color="black", linewidth=1)
ax.set_xticks(x, seasons.value)
ax.set_ylabel("skill vs. naive (%)",
    title="Same model, same predictions – the denominator decides the seasonal story")
ax.legend(fontsize=9)
plt.tight_layout(); plt.show()
```

	value	n	mae	baseline_mae	baseline_rule	skill_vs_naive	skill_vs_pooled_naive
6	Autumn	2079	0.2215	0.2173	predict_zero	-0.0191	0.5118
7	Summer	1276	0.8854	0.9097	predict_zero	0.0268	-0.9517
8	Winter	512	0.3008	0.2764	predict_zero	-0.0880	0.3370



```
In [23]: decile = segmented[segmented.segment == "weight_change_decile"].sort_values("value")
fig, ax = plt.subplots(figsize=(10, 4))
colors = ["#55a868" if s > 0 else "#c44e52" for s in decile.skill_vs_pooled_naive]
ax.bar(decile.value, decile.skill_vs_pooled_naive, color=colors)
ax.axhline(0, color="black", linewidth=1)
ax.set(title="Skill vs. naive by |weight change| decile (pooled bar – see note)",
    xlabel="decile of |actual change|", ylabel="MAE reduction vs. best naive")
plt.tight_layout(); plt.show()
display(decile[["value", "mae", "n", "baseline_rule",
    "skill_vs_pooled_naive", "skill_vs_naive"]].round(4))
```



	value	mae	n	baseline_rule	skill_vs_pooled_naive	skill_vs_naive
9	D1	0.0821	387	predict_zero	0.8191	-9.3879
10	D10	2.9309	387	persistence	-5.4609	0.0758
11	D2	0.0697	387	predict_zero	0.8464	-1.6800
12	D3	0.0895	386	predict_zero	0.8028	-0.9832
13	D4	0.1022	387	predict_zero	0.7748	-0.5752
14	D5	0.1268	387	predict_zero	0.7205	-0.4317
15	D6	0.1538	386	predict_zero	0.6610	-0.2881
16	D7	0.2038	387	predict_zero	0.5507	-0.2815
17	D8	0.2498	386	predict_zero	0.4494	-0.1096
18	D9	0.4996	387	predict_zero	-0.1014	-0.2873

Reading the decile chart. The bars use `skill_vs_pooled_naive`, and that is deliberate. A `|change|` decile is defined by the observed target, so a naive baseline computed *within* a decile is a rule that already knows how large the change was — the `skill_vs_naive` column is printed beside it only to show how far that reframing moves the number. D10 illustrates the problem directly: it is the one decile where the within-segment winner is `persistence` rather than `predict_zero`, and the baseline rule is itself chosen by lowest error on the test rows. For target-defined segments the pooled bar is the appropriate reference; for season and month, which are known before the forecast is made, the within-segment bar in Section 6.1 is.

The shape of the pooled chart is close to arithmetic: small actual changes are easy to beat a fold-wide bar on and large ones are not. It shows where error concentrates, not where the model is skilful.

The table below restates the season rows printed above, with the two skill columns side by side.

Season	MAE	Within-season naive	One fold-wide bar (as published)	Within-season bar
Autumn	0.2215	0.2173	+51.2%	-1.9%
Winter	0.3008	0.2764	+33.7%	-8.8%
Summer	0.8854	0.9097	-95.2%	+2.7%

Seasonal Performance

Seasonal performance was evaluated using season-specific naive baselines to account for differences in the underlying variability of hive weight change throughout the year. Because the final validation fold spans July through December 2022, the evaluation contains summer, autumn, and winter observations but does not include spring.

When performance is evaluated relative to the appropriate within-season naive baseline, model skill is limited across all three represented seasons. Skill scores range from -8.8% in winter to 2.7% in summer, with autumn at -1.9%. These results differ substantially from comparisons based on a single pooled baseline because the magnitude and variability of

daily hive-weight changes differ across seasons. In particular, winter observations exhibit substantially lower absolute variation than summer observations, making a zero-change prediction comparatively competitive during periods of low colony activity.

The seasonal results therefore provide limited evidence that the current daily-grain model consistently improves upon simple baseline predictions within individual seasons. Summer shows a small positive skill score, while autumn and winter do not outperform their respective naive baselines. This finding reinforces the importance of selecting reference predictions that reflect the structure of the evaluation being performed.

6.2 Refinement of Milestone 3 Findings

The expanded dataset and more rigorous validation framework used in Milestone 4 provide additional context for several findings reported in Milestone 3. These differences reflect changes in data coverage, preprocessing, and evaluation methodology rather than direct contradictions of the earlier exploratory analysis.

First, the routine-observation model performance reported in Milestone 3 was calculated after extreme observations had been identified using the observed next-day target. Because extreme-event status would not be known at prediction time, this evaluation represents an upper-bound estimate of routine-day predictability rather than deployable forecasting performance. Under the current framework, regression performance is evaluated on the complete, unfiltered test distribution, which produces more conservative but operationally realistic estimates.

Second, the expanded extreme-event analysis indicates that unusual weight changes cannot be attributed exclusively to colony-level biological behavior. A substantial proportion of extreme observations coincide with data-quality indicators, and the comparison between the publisher's two weight series shows that the majority are affected by their rate-based filtering procedure. At the same time, incomplete and inconsistently recorded beekeeper-event data prevent definitive attribution of individual events, and the event-proximity analysis in Section 3.3 does not by itself support a beekeeper-activity explanation. Milestone 4 therefore treats the extreme-event population as a heterogeneous mixture of biological changes, management activity, and potential measurement artifacts.

Finally, analyses of event proximity and temporal clustering were recomputed at the daily modeling grain used by the predictive models. Earlier exploratory analyses incorporated publisher-defined outlier information derived from higher-frequency observations, which does not correspond directly to the ± 5 kg daily extreme-event definition used for modeling. Aligning these analyses with the final modeling population provides a more consistent basis for interpreting the relationship between extreme observations, beekeeper activity, and predictive performance.

Collectively, these refinements illustrate how conclusions developed during exploratory modeling can change as data coverage expands and validation procedures become more rigorous. The Milestone 4 results therefore build upon the earlier analysis by separating exploratory findings from conclusions that remain supported under the complete dataset and prospective evaluation framework.

6.3 Overall Predictive Performance

Evaluation against the naive baselines indicates that predictive performance at the daily grain remains limited. The single-stage regression models trained on the full target distribution did not improve upon the strongest naive baseline in MAE, with skill scores ranging from approximately -6% to -11%.

The best deployable result came from excluding extreme observations from the regressor's training distribution, which produced roughly a 6% improvement in MAE over the strongest naive baseline. As the ablation in Section 5.3 shows, the two-stage classifier-regressor architecture and a plain regressor trained on routine rows with no gate reach that improvement equally (5.9% and 6.2% respectively). The gain is therefore attributable to the composition of the training set, not to routing predictions with a classifier. That distinction matters for how the result should be described: the useful finding is that extreme observations degrade the regressor when they are included in training, not that a two-stage architecture is required.

The extreme-event classifier produced the strongest evidence of learnable structure. Across the four rolling-origin validation folds, PR-AUC remained approximately 10–17 times greater than the corresponding no-skill rate. This result indicates that extreme weight-change events contain detectable predictive signal, even though acting on that signal through prediction routing did not reduce regression error here. Its value is more plausibly as a standalone detection or alerting output than as a component of a regression pipeline.

Taken together, these findings suggest that next-day changes in daily mean hive weight contain substantial short-term variability that limits regression performance at the daily grain. The results also indicate that predictive information is not distributed uniformly across the target. Detecting extreme events appears to be a more tractable problem than regressing the entire distribution, and should be treated as a separate task rather than a stage within the regression model.

6.4 Temporal Stability of Model Performance

Model performance varied across the four rolling-origin validation folds, indicating that predictive accuracy is sensitive to the temporal period used for evaluation. HistGradientBoosting MAE ranged from approximately 0.43 to 0.63 kg across folds, while R^2 also varied substantially.

This variation should be interpreted in the context of the seasonal structure of hive-weight dynamics. Because each validation fold represents a different temporal window, changes in colony activity, environmental conditions, and the underlying variance of hive-weight change can affect both prediction error and R^2 . Consequently, direct comparison of R^2 across folds should be made cautiously because the metric is normalized relative to the variance of the target within each evaluation period.

The observed fold-to-fold variation reinforces the importance of the rolling-origin validation framework. Rather than relying on performance from a single chronological split, evaluation across multiple temporal windows provides a more representative assessment of model generalization and demonstrates that predictive performance is not constant throughout the study period.

7. Conclusions and Recommendations

7.1 Summary of Findings

The Milestone 4 analysis provides a more comprehensive assessment of next-day hive-weight prediction by expanding the dataset, strengthening temporal validation, evaluating alternative modeling frameworks, and explicitly examining extreme weight-change events.

The strongest deployable regression results came from excluding extreme observations from the regressor's training distribution, which improved MAE by roughly 6% relative to the strongest naive baseline on the unfiltered evaluation data. The two-stage classifier-regressor framework (MAE 0.479 kg, R^2 0.044, skill 5.9%) and a single regressor trained on routine rows with no classifier gate (MAE 0.478 kg, R^2 0.052, skill 6.2%) achieved this equally, so the gain is attributable to the training-set filter rather than to the two-stage architecture. The simpler configuration should be preferred on that basis. In either form the improvement is modest, and the low R^2 indicates that substantial variability in next-day hive-weight change remains unexplained at the daily grain.

Extreme weight-change events demonstrated substantially stronger predictive structure. Across the rolling-origin validation folds, the extreme-event classifier achieved PR-AUC values of 0.17–0.31 compared with no-skill rates of 0.010–0.018. This result indicates that extreme observations contain detectable advance signal despite their low prevalence.

Season-specific evaluation provided little evidence of consistent improvement over appropriate within-season naive baselines. Skill ranged from -8.8% in winter to 2.7% in summer, indicating that apparent seasonal differences in earlier evaluations were largely influenced by baseline selection rather than substantial changes in model performance.

Leave-hives-out evaluation produced MAE broadly comparable to the temporal validation results, but measured against a predict-zero baseline within each held-out block it is approximately level with predicting no change: mean skill of +0.8%, negative in two of five folds. That split also partitions on hive identity only, leaving training and test rows spread across the same period, so it tests transfer across colonies rather than across time. It is consistent with, but does not establish, generalization beyond the specific hives in the training data. Permutation feature importance further demonstrated that recent hive-weight dynamics, particularly previous-day weight change, provide substantially more predictive information than absolute hive-weight level or the evaluated sensor and seasonal variables.

Collectively, these findings indicate that precise next-day regression at the daily grain remains challenging, while detection of unusual weight-change events and modeling of recent weight dynamics provide more promising directions for continued development. Detection is best pursued as a task in its own right, since routing regression predictions on the classifier's output did not improve regression error.

7.2 Practical Implications

Daily Regression

The daily-grain regression results indicate that exact next-day hive-weight change remains difficult to predict with sufficient accuracy for operational decision-making. The strongest deployable framework produced only modest improvement over the naive baseline, suggesting that daily forecasts should not currently be used independently to guide management decisions such as harvest timing.

Seasonal performance should also be interpreted in the context of colony activity. During winter periods of reduced activity, hive weight exhibits comparatively limited day-to-day variation, making a prediction of little or no change a strong naive baseline. The negative winter skill therefore reflects the difficulty of improving upon this strong baseline during relatively stable periods rather than indicating that winter conditions are inherently less predictable.

Weekly Forecasting

Aggregating observations over a longer interval may preserve cumulative changes in colony weight while reducing the influence of short-term measurement variability and day-to-day fluctuation. Weekly forecasting may therefore prove a more appropriate temporal scale for modeling hive-weight trends than next-day prediction, particularly for applications involving cumulative colony weight and harvest forecasting.

No weekly-grain model is evaluated in this notebook, so this is a hypothesis rather than a finding. Section 7.4 sets out the comparison that would test it, which would need to run under the same rolling-origin framework and against within-segment naive baselines before any claim about weekly performance could be supported.

Sensor-Based Extreme-Event Detection

The extreme-event classification results also suggest a potential application for continuous hive monitoring. Rather than relying exclusively on exact next-day weight forecasts, an operational system could use incoming sensor measurements to estimate whether a colony is approaching an unusual weight-change event and generate an alert when predicted risk exceeds a selected decision threshold. Such a system could prioritize sensitivity when the cost of failing to identify a meaningful event is greater than the cost of investigating a false alert.

The current analysis also demonstrates that the definition of an unusual event requires additional study. The ± 5 kg threshold used for continuity with Milestone 3 is an absolute criterion and therefore does not account for differences in typical weight or variability among hives. Hive-relative approaches, such as thresholds based on each colony's historical weight-change distribution, may provide a more appropriate basis for sensor alerting because they identify deviations relative to the expected behavior of the individual hive. However, these thresholds should be evaluated against independently identified biological and management events before being interpreted as operational warning criteria.

This framework could support future development of calibrated sensor-based monitoring in which alert thresholds are selected according to the desired balance between precision and recall. Before such a system could be used for colony-health monitoring, improved event labels would be required to distinguish biologically meaningful changes from beekeeper interventions, harvest activity, and sensor anomalies.

7.3 Scope of Inference

The findings of this study should be interpreted within the population, temporal resolution, and measurement structure represented by the available data. The complete daily archive contains observations from 78 hives in Germany between 2019 and 2022. Although the dataset provides repeated observations across multiple colonies and years, it represents a single geographic setting and citizen-science data collection framework. Consequently, the predictive relationships identified here should not be assumed to generalize directly to colonies managed under different climates, forage conditions, management practices, or sensor configurations.

The unit of analysis also constrains the conclusions that can be drawn. Daily hive-weight values are derived from higher-frequency sensor measurements, which reduces short-term measurement variability but may also obscure biologically or operationally meaningful within-day changes. The results therefore characterize predictability at the daily grain and should not be interpreted as evidence that hive-weight dynamics are inherently unpredictable at finer temporal resolutions.

Finally, inference regarding extreme weight-change events is limited by uncertainty in event attribution. Many observations classified as extreme using the ± 5 kg threshold appear associated with beekeeper handling or other data-quality characteristics, while the available beekeeper logs are incomplete and inconsistently recorded. The analysis can therefore

establish that unusual weight changes contain detectable predictive structure, but it cannot reliably determine whether an individual event represents colony behavior, beekeeper intervention, harvest activity, or sensor-related variation without improved event labels.

7.4 Future Research

Several extensions of the current analysis could clarify the temporal, environmental, and colony-specific factors that determine the predictability of hive-weight dynamics.

First, future work should systematically evaluate temporal resolution and prediction horizon as separate modeling decisions. The source archive contains minute-level observations that may preserve short-duration behavioral, environmental, and management-related signals obscured by daily averaging. Modeling hourly or intermediate-resolution observations could determine whether the limited performance of daily regression results partly from aggregation loss. Separately, cumulative weight dynamics over multi-day windows may contain more predictable structure than next-day changes, because aggregation damps the short-term measurement variability that dominates the daily target. Direct comparison of daily, multi-day, weekly, and longer prediction horizons under a common validation framework could identify the temporal scale at which hive-weight forecasting becomes most informative.

Second, the contribution of environmental predictors should be evaluated through completion of the weather-integration pipeline. Weather data have been acquired for the study locations but are not yet incorporated into the primary feature matrix. Controlled feature-ablation experiments should compare models using weight history alone, weight history with internal hive-sensor measurements, and the complete feature set including external weather conditions. Evaluating each feature group under identical rolling-origin validation folds would determine whether environmental variables provide incremental predictive information beyond that already contained in recent hive-weight dynamics.

Third, future analysis should investigate redundancy among engineered weight-history predictors. Lagged weights, rolling statistics, weight-change variables, and trend features represent related aspects of recent colony behavior and may contain overlapping information. Feature-group ablation experiments could evaluate model performance after systematically removing or reducing these predictor families. This would help determine which representations provide independent predictive value, improve interpretation of permutation importance, and potentially produce a more parsimonious model without sacrificing out-of-sample performance.

Fourth, hive-relative representations of weight and unusual weight-change events should be investigated. Colonies vary substantially in baseline weight and weight-change variability, making a single absolute threshold such as ± 5 kg potentially different in biological significance across hives. Future models could evaluate changes relative to each colony's historical distribution, baseline weight, or variability and compare these approaches with the current absolute definition. Any resulting threshold should be validated against independently identified biological and management events before being interpreted as an operational alert criterion.

Finally, the strong relative performance of the extreme-event classifier supports further investigation of sensor-based event detection as a distinct prediction problem. Future work should improve event labels to distinguish beekeeper interventions, harvesting, sensor anomalies, and biologically driven colony changes and then evaluate classification thresholds according to the desired balance between precision and recall. This could determine whether continuous hive-weight measurements are better suited to detecting meaningful departures from expected colony behavior than to producing precise next-day weight forecasts.

7.5 Limitations

Several limitations should be considered when interpreting the results of this analysis. First, the primary modeling target is derived from the publisher's `weight_kg` series, which retains abrupt changes that may result from beekeeper handling, harvesting, sensor artifacts, or genuine colony behavior. Although the publisher provides an alternative cleaned weight series, that series removes many abrupt changes and may also eliminate biologically meaningful events. The current analysis therefore retains the original weight series for continuity while explicitly evaluating the influence of extreme observations.

Second, the ± 5 kg definition of an extreme weight change was retained for comparability with Milestone 3 but represents an absolute threshold applied across colonies with substantially different weight distributions. This criterion may classify equivalent absolute changes differently in terms of their biological significance across hives. More broadly, the use of absolute hive-weight and weight-change measurements may not fully account for differences in colony size and baseline hive weight, suggesting that future modeling should evaluate hive-relative or normalized representations of weight

dynamics. The threshold-sensitivity analysis indicates that hive-relative definitions may provide a more appropriate basis for future event detection, although these alternatives require further biological validation.

Third the engineered feature set also contains several related measures of recent hive-weight history, including lagged weights, rolling averages, and weight-change features. Although permutation importance was used to evaluate predictor contributions, the current analysis did not systematically quantify multicollinearity or determine whether correlated weight-history features provide independent predictive information. Consequently, individual feature-importance rankings should be interpreted cautiously, and future feature-ablation analysis should evaluate whether a reduced set of weight-history predictors can achieve comparable out-of-sample performance.

Fourth, two evaluation results carry qualifications that limit their weight. The event-proximity comparison in Section 3.3 is sensitive to the choice of denominator: computed over rows that carry a logged event it suggests associations with feeding and treatment, but computed over all rows in each group the treatment association disappears and the feeding association rests on 35 extreme observations. The unit normalisation applied to the source event-distance fields also resolves fewer hives than its summary table indicates. Separately, the leave-hives-out evaluation holds out colonies but not time periods, and once a within-block naive baseline is attached it shows approximately no improvement over predicting zero change; it should not be read as establishing generalization to new colonies.

Finally, the current primary modeling workflow does not yet incorporate the available weather variables into the complete feature matrix. The resulting model comparisons therefore evaluate weight-history and available hive-sensor information rather than the full set of environmental predictors originally proposed for the project. In addition, the citizen-science origin of the dataset introduces variation in sensor availability, beekeeper record completeness, and measurement practices that cannot be fully controlled retrospectively.

Appendix A — honey_model schema

```
In [24]: schema = pd.DataFrame({"column": model_df.columns, "dtype": [str(t) for t in model_df.dtypes]})
schema["forbidden_as_feature"] = schema.column.isin(features.FORBIDDEN_COLUMNS)
display(schema)
```

	column	dtype	forbidden_as_feature
0	hive_id	int32	False
1	measurement_date	datetime64[ns]	False
2	end_of_day_weight_kg	float64	False
3	previous_observation_date	datetime64[ns]	False
4	next_observation_date	datetime64[ns]	True
...	...	...	...
67	nearest_swarming_event_days	float64	True
68	nearest_treatment_event_days	float64	True
69	nearest_died_event_days	float64	True
70	nearest_queencell_event_days	float64	True
71	season	object	False

72 rows × 3 columns

Appendix B — Reproduction checklist

"Shown here" means the notebook demonstrates the check when it executes. "Enforced in code" means the pipeline raises if the check fails, but this notebook does not exercise it directly.

Check	Where	Status
Clean-checkout run with no network, no VPN, no DuckDB file	this notebook	shown here — Section 1 loads only data/honey_model.parquet

Check	Where	Status
Library versions that produced the stored outputs	Section 1	shown here
Published shape asserted on load — 26,215 rows, 78 hives, no duplicate hive-days	<code>data.load_model_table</code>	enforced in code
Rebuild parity — SQL pipeline reproduces the published table	<code>scripts/build_honey_model.py</code>	enforced in code, not re-run here
No leakage — forbidden columns rejected, with a negative test	Section 2.6	shown here
Split integrity — train ends strictly before test begins, every fold	Section 4.1	shown here
Baselines beaten (or not) — skill score on every model	Sections 4.3, 5.1	shown here
Gate ablation — does the two-stage routing beat a routine-trained regressor?	Section 5.3	shown here
Two-stage honesty — unfiltered metrics present, oracle numbers labelled	Section 5.3	shown here
Leave-hives-out scored against a within-block naive baseline	Section 5.5	shown here
Segmented tables — season and change decile displayed; month and hive computed into <code>results/segmented.csv</code> but not shown	Section 6.1	partially shown
Cross-validation — rolling-origin, mean $\pm$ sd	Sections 4.2, 5.1	shown here